

Operating Systems and System Programming

Practical – 2

Task-1

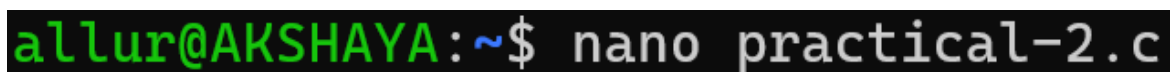
Aim:

To develop a C program using the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another, and to use the `strace` utility to trace and analyze the system calls generated by `cat sample.txt`, thereby understanding the interaction between user space and kernel space.

Algorithm:

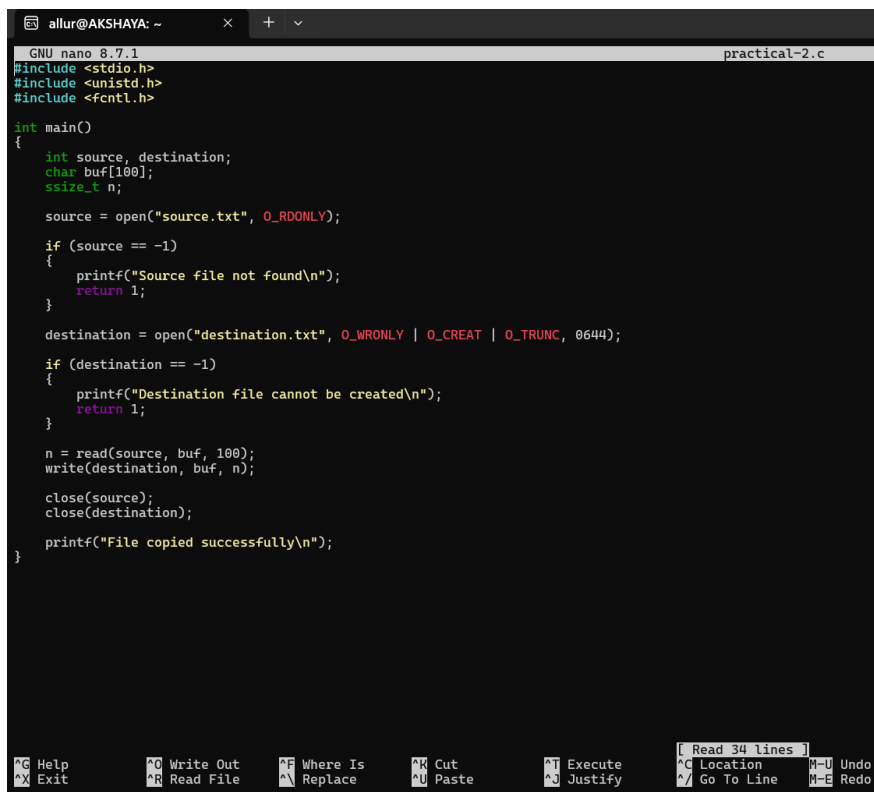
1. Start the program.
2. Open the source file using `open()` in read-only mode.
3. Open or create the destination file using `open()` in write mode.
4. Read data from the source file using `read()` into a buffer.
5. Write the data from the buffer to the destination file using `write()`.
6. Repeat the read and write operations until the entire file is copied.
7. Close both files using `close()`.
8. Execute `strace cat sample.txt` in the terminal.
9. Observe and analyze the system calls such as `execve()`, `openat()`, `read()`, `write()`, and `close()`.
10. Relate these system calls to the corresponding kernel services and user-space/kernel-space transitions.
11. Stop.

Step-1:



```
allur@AKSHAYA:~$ nano practical-2.c
```

Step-2: Program:



```
GNU nano 8.7.1 practical-2.c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    int source, destination;
    char buf[100];
    ssize_t n;

    source = open("source.txt", O_RDONLY);

    if (source == -1)
    {
        printf("Source file not found\n");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1)
    {
        printf("Destination file cannot be created\n");
        return 1;
    }

    n = read(source, buf, 100);
    write(destination, buf, n);

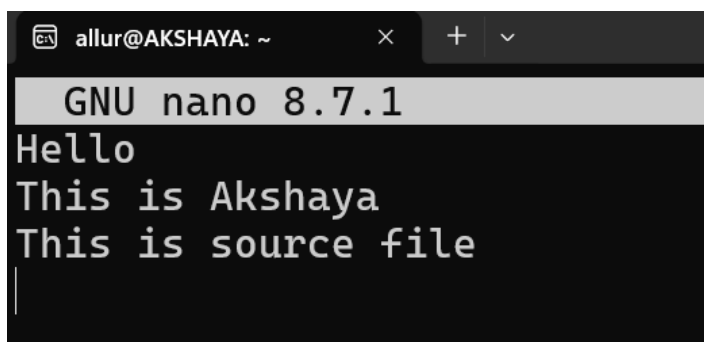
    close(source);
    close(destination);

    printf("File copied successfully\n");
}
```

Step-3: Compilation and Execution

```
allur@AKSHAYA:~$ nano practical-2.c
allur@AKSHAYA:~$ gcc practical-2.c
allur@AKSHAYA:~$ ./a.out
Usage: ./a.out <source_file> <destination_file>
allur@AKSHAYA:~$
```

Step-4: Contents of source.txt:



```
GNU nano 8.7.1
Hello
This is Akshaya
This is source file
```

Step-5: Contents of destination.txt:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 destination.txt  
Hello  
This is Akshaya  
This is source file
```

Task-2

Command:[illegible]

```
mmap(NULL, 0x79bfdb95f800, MAP_PRIVATE|MAP_SHARED|PROT_READ|PROT_WRITE, 0, 0) = 0x79bfdb95f800
mmap(0x79bfdb95f800, 5070904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 1, 0x3080) = 0x79bfdb95f800
mmap(0x79bfdb95f800, 176128, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7f000) = 0x79bfdb95f800
mmap(0x79bfdb95f800, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 5, 0xa9000) = 0x79bfdb95f800
close(2) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x79bfdbc87600
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x79bfdbc84000
arch_prctl(ARCH_SET_FS, 0x79bfdb95d000) = 0
set_tid_address(&0x79bfdb95c20) = 1731
set_robust_list(&0x79bfdb94e00, 20) = 0
rseq(&0x79bfdb94e00, 0x20, 0, 0x30303030) = 0
mprotect(0x79bfdb94e000, 13304, PROT_READ) = 0
mprotect(0x79bfdb94e000, 40896, PROT_READ) = 0
mprotect(0x79bfdb94e000, 40896, PROT_READ) = 0
mprotect(0x79bfdb94e000, 40896, PROT_READ) = 0
mprotect(0x79bfdb94e000, 40896, PROT_READ) = 0
mprotect(0x79bfdb94e000, 1462272, PROT_READ) = 0
mprotect(0x79bfdb94e000, 8192, PROT_READ) = 0
prctl(PR_SET_PDEATHSIG, SIGKILL, 0, 0, 0) [rin_cur=102+1020, rin_max=NLMGR_INFINITY] = 0
getrandom("x79\x0f\xa9\xdd\x01\x31\xff\x08", 8, GRND_NONBLOCK) = 0
munmap(0x79bfdba10000, 20887) = 0
stratix("/sys/fs/selinux", {f_type=VFAT_MAGIC, f_size=0, f_blocks=0, f_bfree=0, f_bavail=0, f_files=0, f_ffree=0, f_fsid=[val=[0x120c7332, 0x3643ab2], f_name_len=255, f_size=4096, f_blocksize=1024, f_namelen=255, f_frsize=4096, f_files_per_block=1024}], 0) = -1 EISDIR (No such file or directory)
brk(NULL) = 0x5f9f5ab57000
brk(&0x5f9f5ab5000) = 0x5f9f5ab5000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
fstat(C, {st_mode=S_IFREG|0444, st_size=...}) = 0
read(C, "nodev\tmodevt\nttaps\nnodev\tbdi...", 1020) = 442
lseek(C, 0, SEEK_CUR) = 442
lseek(C, 0, SEEK_SET) = 416
close(C) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
fstat(C, {st_mode=S_IFREG|0444, st_size=...}) = 0
read(C, "name /usr/lib/modules/6.18.32-2...., 1020) = 1024
read(C, " 0 0 &udevpts /dev/pts devpts rw..., 1020) = 1024
read(C, "access_client,nsize=65536,trans..., 1020) = 1024
read(C, "id.service taps ro,nosuid,nodev..., 1020) = 250
read(C, "..., 1020) = 0
close(C) = 0
accept("/etc/selinux/config", F_OK) = -1 ENOENT (No such file or directory)
fcntl(0, F_GETFD) = 0
fcntl(0, F_GETFD) = 0
fcntl(0, F_GETFD) = 0
rt_sigaction(SIGPIPE, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 0) = 0
poll([fd=0], [fd=0, events=[]], [fd=0, events=[]], 3, 0) = 0 (timeout)
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=SA_RESTORER, sa_restorer=0x79bfdba4fc00}, [{sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 0]) = 0
prctl(PR_SET_PDEATHSIG, SIGKILL, 0, 0, 0) [rin_cur=102+1020, rin_max=NLMGR_INFINITY] = 0
```

[illegible]

Sequence of Execution

User enters: cat sample.txt

↓

execve()

↓

Execute cat program

↓

openat()

↓

Open sample.txt

↓

read()

↓

Read file contents

↓

write()

↓

Display contents on terminal

↓

close()



Close file



exit_group()



Process terminates

Result

The file-copy operation was successfully implemented using Linux system calls, and the strace utility was used to trace the system calls generated by cat sample.txt. The experiment demonstrated how user-space programs interact with the Linux kernel through system calls for file access, reading, writing, and closing operations.